

Failure (bad key + cost comparison) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 2 depth levels: **Basic** (the plain idea) and **Intermediate** (proper Python). Read Basic first even if you already know Python — it's the fastest way to spot exactly what Intermediate adds.

- [Hint 1 — The two parts, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The two parts, and the exact pieces

Basic Version

Two separate things to try here: 1. Use a fake API key on purpose and see exactly what error comes back. 2. Ask the model the same question twice — once with a short system prompt, once with a long one — and compare how many tokens each one used.

`OpenAI(api_key="fake-key-123")` — make a client with a wrong key on purpose. Wrap the call in `try/except`, catching the specific auth error, not a bare `except`. `response.usage` has the token counts on it after any real, successful call.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Part 1 tests that you catch `openai.AuthenticationError` *specifically*, not just any exception — because a bare `except Exception` would also hide a real bug in your own code behind the same generic message, making it much harder to tell "bad key" apart from "I broke something."

`OpenAI(api_key="sk-fake-key-123")` — passing a bad key directly to the client constructor (instead of relying on the real one from `.env`) forces an authentication failure on the very next call.

Part 2 uses `response.usage` — every response object carries back the exact token counts OpenAI billed you for, split into `prompt_tokens`, `completion_tokens`, and `total_tokens`. Comparing this number between a 1-sentence and a 20-sentence system prompt turns "shorter prompts cost less" from a rule you were told into a number you measured yourself.

Difference between Basic and Intermediate: Basic measures a single call's token count directly. Intermediate splits the comparison into named, typed functions so each concern (bad key, cost) is independently reusable.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
part 1:
    make a client with a fake key
    try to send a message
    except the specific "bad key" error:
        print a clean message

part 2:
    ask the same question with a short system prompt, note the token count
    ask the same question with a long system prompt, note the token count
    compare the two numbers
```

Here's almost the whole thing for part 1:

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

bad_client = OpenAI(api_key="sk-fake-key-123")
try:
    bad_client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": "hi"}],
    )
except openai.AuthenticationError:
    print("Login failed – check your API key.")
```

Expected output: Login failed – check your API key. Now write part 2 yourself: two calls, one short system prompt, one long, comparing `.usage.total_tokens`.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
function try_bad_key():
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try: send a message
    except openai.AuthenticationError: print a clean message

function compare_prompt_cost(question):
    short = call with a short system prompt
    long = call with a long system prompt
    print short.usage.total_tokens
    print long.usage.total_tokens
```

```
# auth_and_cost_practice.py
import openai
from openai import OpenAI

client = OpenAI()

def try_bad_key() -> None:
    bad_client = OpenAI(api_key="sk-fake-key-123")
    try:
        bad_client.chat.completions.create(
            model="gpt-4o-mini",
            messages=[{"role": "user", "content": "hi"}],
        )
    except openai.AuthenticationError:
        print("Login failed – check your API key.")

def compare_prompt_cost(question: str) -> None:
    short_prompt = "Be helpful."
    long_prompt = "You are a friendly, detailed, thorough assistant who explains

    short = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "system", "content": short_prompt}, {"role": "user", "
    )
    long = client.chat.completions.create(
        model="gpt-4o-mini",
```

```
        messages=[{"role": "system", "content": long_prompt}, {"role": "user", "c
    )

    print(f"Short system prompt: {short.usage.total_tokens} tokens")
    print(f"Long system prompt: {long.usage.total_tokens} tokens")
```

Call both functions and confirm the long prompt uses noticeably more tokens, then compare against the [Solution](#).

Difference between Basic and Intermediate: Basic and Intermediate both measure and compare token counts for exactly 2 calls, side by side. Intermediate wraps each concern in its own named, typed function instead of one flat script.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)